
The Instrument Panel for Adversarial Robustness: Measuring, Budgeting, and Controlling the Shadow Price

Guangyu Wang
billgywang@gmail.com

Abstract

A companion paper (Paper A) shows that the marginal price of adversarial robustness—the subgradient $\lambda_\varepsilon = \partial L^*/\partial \varepsilon$ of a constrained value function in the adversarial radius ε —has a floor whose correct pricing unit is the *effective support* $k_\varepsilon = (\|w^*\|_1/\|w^*\|_2)^2$ of the robust-optimal discriminant, $\lambda_\varepsilon \geq c_M \sqrt{k_\varepsilon}/\sigma$. That result says the price *exists*; it does not, by itself, tell a practitioner what the price *is* on their model, how much robustness to *buy*, or how to *steer* the training budget. This paper closes that gap with an open-source toolkit, **shadowprice**, organized as three layers—**measure**, **decide**, **control**—and a validation protocol. The measurement layer reads the price from one backward pass per batch, with no adversarial inner loop ($\lambda_\varepsilon \approx \mathbb{E}\|\nabla_x \ell\|_1$), brackets the curvature correction κ_∞ , tracks the effective dimension k_ε , and extrapolates the price without a *per-radius* robust retrain, accurate up to a *measurement wall* (exact in the linear model, approximate in general). The decision layer computes the certification radius and the economically-optimal radius ε^{**} . The control layer drives the budget to ε^{**} in closed loop. We unit-test the instruments against closed forms (Stage A, 13/13 checks), demonstrate them on a small trained model with a real measured breach curve (Stage A.5), and specify a Stage-B protocol with explicit falsification conditions for deep networks at scale. The toolkit’s core is framework-agnostic; heavy dependencies are optional.

1 Introduction

Adversarial robustness is not free, and the cost is not a modeling detail one can wish away. Paper A ([Companion paper \(Paper A\), 2025](#)) makes this precise: treating the adversarial radius ε as a one-dimensional budget of a constrained value function $L^*(\varepsilon)$, the *marginal price of robustness* is the subgradient $\lambda_\varepsilon = \partial L^*/\partial \varepsilon$, and it obeys a dimensional floor

$$\lambda_\varepsilon \geq c_M \frac{\sqrt{k_\varepsilon}}{\sigma}, \quad k_\varepsilon = \left(\frac{\|w^*\|_1}{\|w^*\|_2} \right)^2, \quad (1)$$

where w^* is the robust-optimal discriminant and k_ε —its *effective support*, the participation ratio of the soft-thresholded active set—is the true pricing unit, specializing to $\sqrt{d}$ (dense), $\sqrt{k}$ (sparse), or $\sqrt{r_{\text{eff}}}$ (low effective rank) as special cases.

Equation (1) is an existence statement. For an engineer it raises three operational questions it does not answer:

1. **Measure.** What is λ_ε on *my* model, right now, and can I trust the measurement?
2. **Decide.** How much robustness should I buy—what is the economically-optimal radius?
3. **Control.** How do I steer the training budget toward that optimum without re-running everything?

This paper answers all three with a small, validated, open-source toolkit, **shadowprice**, and a companion protocol. Our contribution is not new theory—the theorems live in Paper A—but the *operationalization*: a set of instruments, each tied to a closed form of Paper A, each with a cheap estimator, and each validated. Concretely:

- a **price meter** that reads $\lambda_\varepsilon \approx \mathbb{E}\|\nabla_x \ell\|_1$ from one backward pass per batch (no adversarial inner loop), with error bars and a δ^{-3} sampling budget (§3);
- a **curvature early-warning** κ_∞ that flags when the first-order price proxy is untrustworthy (§3);
- an **effective-support diagnostic** k_ε as a per-example effective dimension, tracked over training (§3);
- a **retraining-free proxy** that extrapolates the price across ε up to a *measurement wall* (§3);
- a **decision calculus** for the certification radius $\varepsilon_{\text{cert}}$ and the optimal radius ε^{**} (§4);
- a **closed-loop controller** that drives the budget to ε^{**} (§5).

We validate every instrument against Paper A’s closed forms on a synthetic linear–Gaussian model (Stage A, §8), and lay out a Stage-B protocol with falsification conditions for real networks (§8). The library is described in §9. We position against the existing robustness-tooling landscape—which evaluates and attacks, but does not *price*—in §10.

2 Background: the resource-variational principle

We keep this section self-contained: the three closed forms the instruments rely on are derived here in a few lines, so the paper stands without Paper A ([Companion paper \(Paper A\), 2025](#)) (which supplies the general theory and the floor’s constant). For a binary Gaussian-mixture model $x = y\mu + z$, $z \sim N(0, \sigma^2 I)$, and a linear discriminant w facing an ℓ_∞ adversary of radius ε , the robust risk is $R(\varepsilon; w) = \Phi(t(w))$ with standardized margin $t(w) = (\varepsilon\|w\|_1 - \langle w, \mu \rangle) / (\sigma\|w\|_2)$, and the marginal price is

$$\lambda_\varepsilon = \frac{\partial R}{\partial \varepsilon} = \frac{\sqrt{k_\varepsilon}}{\sigma} \phi(t), \quad k_\varepsilon = (\|w\|_1 / \|w\|_2)^2. \quad (2)$$

Derivations (self-contained). All three closed forms follow from one computation. **(a) Risk.** The worst-case ℓ_∞ perturbation of radius ε shifts the clean margin $y\langle w, x \rangle$ by its support function $-\varepsilon\|w\|_1$ (the ℓ_1 dual of the ℓ_∞ ball, attained at $\delta = -\varepsilon y \text{sign}(w)$). Misclassification occurs when $\langle w, \mu \rangle + y\langle w, z \rangle < \varepsilon\|w\|_1$; since $y\langle w, z \rangle \sim N(0, \sigma^2\|w\|_2^2)$, this probability is $\Phi(t(w))$. **(b) Price.** Differentiating $R = \Phi(t)$ in ε (with $\partial t / \partial \varepsilon = \|w\|_1 / (\sigma\|w\|_2)$) gives (2); the prefactor is exactly $\|w\|_1 / \|w\|_2 = \sqrt{k_\varepsilon}$. **(c) Soft-threshold optimum.** Minimizing $t(w)$ over $\|w\|_2 = 1$ is $\min_w \varepsilon\|w\|_1 - \langle w, \mu \rangle$; aligning signs ($\text{sign } w_i = \text{sign } \mu_i$) and maximizing the linear functional over the positive-orthant sphere yields $w_i^* \propto \text{sign}(\mu_i)(|\mu_i| - \varepsilon)_+$ —the soft-threshold, whose active set has participation ratio k_ε . The general (non-Gaussian, log-concave) versions replace ϕ by the margin density p_S and are proved in Paper A; the toolkit uses only these three lines plus that density generalization. Three facts then drive the toolkit. **(i) The pricing unit is k_ε :** the ℓ_∞ attack induces ℓ_1 dual geometry, whose prefactor $\|w\|_1 / \|w\|_2 = \sqrt{k_\varepsilon}$ is the effective dimension being taxed. **(ii) A cheap proxy:** as $\varepsilon \rightarrow 0$, $\lambda_\varepsilon \rightarrow \mathbb{E}\|\nabla_x \ell\|_1$, obtainable from one backward pass, with the finite- ε correction controlled by the ℓ_∞ curvature $\kappa_\infty(x) = \max_{s \in \{\pm 1\}^d} |s^\top \nabla^2 f(x) s|$ via $|\lambda_\varepsilon - \hat{\lambda}_\varepsilon| \leq \varepsilon \mathbb{E}[\kappa_\infty] + O(\varepsilon^2)$. **(iii) The floor holds in a margin-density window** $|t| \leq M$, where $\phi(t) \geq \phi(M) =: c_M$; outside it (the over-confident regime, $|t|$ large) the price is negligible and the measurement enters the tail. These three facts become, respectively, the effective-support diagnostic, the price meter, and the curvature/collapse checks.

The train–inference bridge. A fourth fact makes the toolkit coherent across the training/deployment boundary. Paper A’s *readout operator* shows that the training-side costate (the sensitivity of the value to the training control) and the deployment-side robust price are two readouts of a single subgradient object ∂L^* —“change the axis, not the machine.” Operationally this means a quantity measured *during training* (the input-gradient dual norm) predicts the *deployment* price up to the $O(\varepsilon)$ curvature term. This is what licenses (i) the retraining-free proxy—measuring once and extrapolating—and (ii) the train–inference marginal gap as a consistency observable (O6): its size is exactly the $\varepsilon \mathbb{E}[\kappa_\infty]$ correction, so a gap larger than the curvature bound signals a broken measurement rather than a genuine train/deploy discrepancy.

Imported assumptions from Paper A. The toolkit inherits, without re-proving, five results/conditions; where they fail, the corresponding instrument is not guaranteed. **(A1)** the $\sqrt{k_\varepsilon}$ price floor and its constant (Paper A, Thm 4.1); **(A2)** the margin-density generalization $\lambda_\varepsilon = (\sqrt{k_\varepsilon}/\sigma) p_S(t)$ for symmetric log-concave noise (used by the collapse check outside the Gaussian case); **(A3)** envelope/Danskin regularity—the value function’s subgradient is single-valued a.e., so the price is well defined; **(A4)** the local-linearization validity for a C^2 classifier, i.e. the price transfers through $w = \nabla f(x_0)$ within the κ_∞ -bounded neighborhood (not globally); **(A5)** the reduced-form deployment economics of Prop. 8.4— P_{breach} is exogenous, monotone, and supplied by the threat model. A1–A3 are proved in Paper A; A4–A5 are modeling choices this paper adopts and tests empirically only at Stage A.5.

3 The Instrument Panel (measurement)

The measurement layer turns the closed forms above into cheap, validated readouts. Throughout, the library exposes a framework-agnostic core: estimators consume a user-supplied $\text{grad_fn}(x) \rightarrow \nabla_x \ell$ and, optionally, a $\text{risk_fn}(\text{eps}) \rightarrow R$, so numpy, PyTorch, or JAX all work.

Instrument 1: the price meter. The leading term of (2) is the gradient-norm proxy $\hat{\lambda}_\varepsilon = \mathbb{E} \|\nabla_x \ell\|_1$, a single backward pass per batch (no adversarial inner loop); the *estimate* still needs a batch of examples for a stable average, quantified by its bootstrap CI. The library averages it over a batch and returns bootstrap error bars (`price_gradnorm`). For a direct finite- ε reading it differentiates a Monte-Carlo risk by central differences (`price_finite_diff`). Because that risk is itself sampled, the derivative estimator has bias $\sim h^2$ and variance $\sim \sigma^2/(nh^2)$; balancing them gives the characteristic δ^{-3} *budget*—to reach derivative accuracy δ one needs $n \sim \delta^{-3}$ samples—exposed by `fd_budget`. Panel (a) of Figure 1 shows the finite-difference meter tracking the closed form (2) to machine precision.

Cookbook. Use the one-pass proxy for *continuous* monitoring (every few hundred steps, it is essentially free) and the finite-difference meter for *periodic* ground-truth checks. Size the batch so the bootstrap CI is tight—as a rule of thumb, aim for a 95% interval narrower than 10% of the mean before acting on a reading; the input-gradient ℓ_1 norm concentrates quickly, so a few hundred examples usually suffice. When you do take a finite difference, get the step from `fd_budget` rather than guessing: chasing a tiny accuracy δ is expensive ($n \sim \delta^{-3}$), so pick the loosest δ your decision can tolerate. The most common pitfall is measuring in the *over-confident regime*: if $|t|$ is large the price is genuinely near zero and the reading, though correct, is uninformative—confirm you are inside the margin-density window ($|t| \lesssim M$, e.g. robust accuracy bounded away from 0 and 1) before trusting a small price.

Instrument 2: curvature early-warning. The first-order proxy is trustworthy only where the $O(\varepsilon)$ curvature correction is small. Computing κ_∞ exactly is a Boolean quadratic program, but the bridge only needs a computable *upper* bound. The library provides a dependency-free bound $\sum_{ij} |H_{ij}|$, and—when `cvxpy` is present—the tighter SDP relaxation $\max \langle H, X \rangle$ s.t. $X \succeq 0$, $\text{diag}(X) = 1$ (over both signs, so it bounds $\max |s^\top H s|$), paired with a random-sign lower bound (`kappa_infty`). A large upper/lower bracket ratio is the early-warning signal that the proxy may be unreliable at the current ε .

Cookbook. Do not compute κ_∞ every step—the SDP is $O(d^2)$ variables and is meant for *periodic audits*. For continuous monitoring use the dependency-free $\sum_{ij} |H_{ij}|$ bound (or a Hutchinson trace estimate for large d), and reserve the SDP for occasional tightening. Read the bracket as a traffic light: a ratio near 1 means the first-order proxy is trustworthy and you can price at the current ε ; a ratio $\gg 1$ means either shrink ε until the $O(\varepsilon)$ correction is negligible, or fall back to the finite-difference meter. For ReLU networks the Hessian is a.e. zero and the curvature concentrates on activation boundaries; estimate κ_∞ from the gradient jump across the nearest boundaries (Paper A, ReLU remark) rather than from a spectral norm. A practical default: audit κ_∞ whenever the collapse residual (Instrument 4) drifts, since curvature is one of the things that inflates it.

Instrument 3: the effective-support diagnostic. The pricing unit $k_\varepsilon = (\|w\|_1/\|w\|_2)^2$ of an input gradient is a cheap per-example *effective dimension* in $[1, d]$: 1 for a localized gradient, d for a fully delocalized one (`participation_ratio`). Tracked over training (`SupportTracker`), it reveals how the dimensional tax

evolves. Panel (b) of Figure 1 shows k_ε of the soft-threshold discriminant contracting as ε grows and coordinates drop out.

Cookbook. Track the ratio k_ε/d over training as an effective-dimension health metric (**SupportTracker** logs the per-batch mean and 10/90 percentiles). Interpret the trend, not the instantaneous value: $k_\varepsilon/d \rightarrow 1$ means the input gradient is fully delocalized and you are paying the full $\sqrt{d}$ tax; $k_\varepsilon/d \ll 1$ means the model concentrates its sensitivity on a few coordinates—the tax is lower, but check that this is genuine low effective rank and not a brittle reliance on a handful of features (cross-reference with the price and with held-out robustness). A useful sanity check is that $\sqrt{k_\varepsilon}$ should predict the measured price factor $\lambda_\varepsilon \sigma / \phi(t)$ (Instrument 4); a persistent mismatch means the gradient participation ratio is not the operative pricing unit, which itself is diagnostic (e.g. strong feature covariance/coherence, Paper A).

Instrument 4: the dimensionless-collapse check. In general (Paper A’s log-concave floor, not just the Gaussian case) the price factorizes as $\lambda_\varepsilon = (\sqrt{k_\varepsilon}/\sigma) p_S(t)$, where p_S is the density of the standardized margin $t = y\langle w, x \rangle / (\sigma \|w\|_2)$; the Gaussian ϕ is the linear-model special case. The residual

$$\rho_{\text{collapse}} = |\lambda_\varepsilon \sigma / p_S(t) - \sqrt{k_\varepsilon}| / \sqrt{k_\varepsilon} \quad (3)$$

is a scale-invariance self-check: it is ≈ 0 when the measurement is in the validated regime and inflates otherwise. **Estimating p_S in real networks.** There is no closed-form ϕ for a real model, so p_S is estimated *empirically*: collect the per-example margins (the logit gap $yf(x)$, or the standardized t), and form a kernel density estimate over a batch (**estimate_margin_density**). Equivalently, since the ℓ_∞ attack shifts each margin by $\varepsilon \|\nabla_x f\|_1$, the price obeys $\lambda_\varepsilon \approx \bar{g} p_M(\varepsilon \bar{g})$ with $\bar{g} = \mathbb{E} \|\nabla_x f\|_1$ and p_M the KDE of the raw margins—a fully model-agnostic form of (3) that needs no distributional assumption. On the trained MLP of §7 this empirical prediction matches the measured price to within $\sim 30\%$ (median, in-regime), the residual reflecting gradient-margin heterogeneity absent in the linear model. It requires no ground truth beyond the model’s own readouts, and is observable O3 of the Stage-A protocol.

Cookbook. Run the collapse residual as an always-on consistency monitor—it is the cheapest way to know whether *any* of your price readings can be trusted, because it cross-checks the meter, the effective-support diagnostic, and the operating point against a single closed-form identity. A residual below $\sim 5\%$ is healthy *when a closed form applies* (linear/Gaussian regime); with the empirical KDE margin density on a real network the intrinsic floor is higher, so $\sim 30\text{--}40\%$ is the healthy range (Table 1). What matters is the *trend*: a residual rising well past its regime’s floor is a switch that trips before your prices become wrong, and it localizes the fault: it inflates when you leave the margin-density window (over-confident $|t|$), when curvature is large (audit κ_∞), or when the noise is far from log-concave (the identity’s assumption). Treat an inflating residual as a stop sign: re-establish the regime before acting on ε^{**} or the controller.

Instrument 5: the retraining-free proxy. Estimating λ_ε at a target radius normally requires a model *retrained* at that ε . The retraining-free proxy avoids this: it measures the input-gradient geometry once (a single batch of backward passes, no *per-radius* robust retrain) and extrapolates the price across ε using the soft-threshold structure of w^* , $w(\varepsilon) \approx \text{soft}_\varepsilon(w_0)$ and $\lambda_\varepsilon \approx (\sqrt{k_\varepsilon(w(\varepsilon))}/\sigma) \phi(t(\varepsilon))$ (**RetrainFreeProxy**). The extrapolation is *exact in the linear model and approximate in general*; it is reliable while the active support is stable and the operating point stays in the margin-density window, and breaks past the *measurement wall*—the radius at which the support collapses or $|t|$ exits the window. The proxy flags points beyond the wall (Figure 2b).

Cookbook. Calibrate once at a cheap operating point (a single clean-model backward pass), then read off the whole λ_ε curve without retraining—this is what makes budgeting affordable. Respect the wall: never act on an extrapolation flagged **beyond_wall**; past it, pay for an actual robust probe instead. Set the two wall knobs to your risk tolerance: M (the margin-density half-width) guards the tail regime, and **min_active_frac** guards support collapse—tighten them if a wrong price is costly, loosen them to extrapolate further at your own risk. Re-calibrate whenever the model changes materially (a training phase transition, a fine-tune, a distribution shift), since the proxy inherits the geometry of the snapshot it was measured on. In practice the wall lands where the surviving support becomes noise-sensitive, so a widening bootstrap CI on the gradient measurement is an early hint that the wall is near.

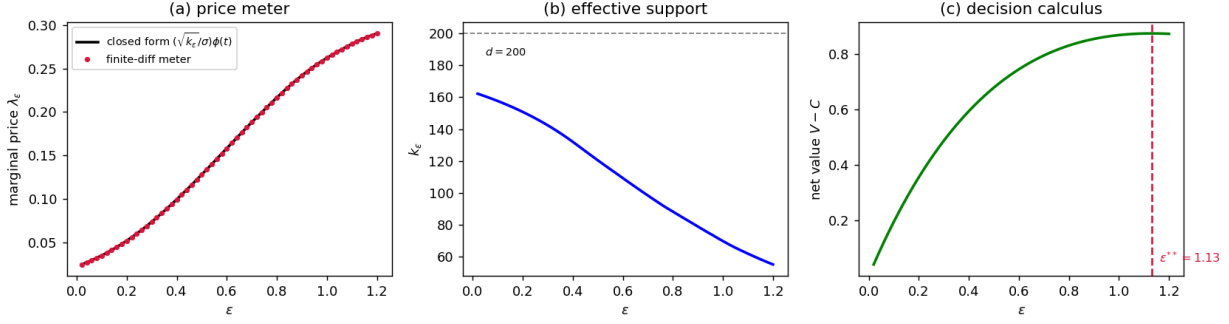


Figure 1: The instrument panel on the Paper A linear-Gaussian model. **(a)** The finite-difference price meter (points) tracks the closed form $(\sqrt{k_\varepsilon}/\sigma)\phi(t)$ (line) to machine precision (mean $|\cdot| \approx 10^{-13}$). **(b)** The effective support $k_\varepsilon(\varepsilon)$ of the soft-threshold discriminant contracts from ~ 160 toward small values as coordinates drop out ($d = 200$). **(c)** The decision calculus: net value $V - C$ and the interior optimal radius ε^{**} .

4 The Decision Calculus (decision)

Measurement feeds decision. Two radii matter. The **certification radius** $\varepsilon_{\text{cert}} = \langle w^*, \mu \rangle / \|w^*\|_1$ (**cert_radius**) is where the robust risk reaches $\frac{1}{2}$; beyond it the classifier is worse than chance and, under the fixed binary-channel reduction, carries no information. The **optimal robust radius** ε^{**} answers “how much to buy.” With a deployment value $V(\varepsilon) = K(1 - P_{\text{breach}}(\varepsilon))$ (K the single-breach loss) and a robustness cost $C(\varepsilon) = \int_0^\varepsilon \lambda_{\varepsilon'} d\varepsilon'$ whose marginal is the shadow price, $\varepsilon^{**} = \arg \max_\varepsilon [V - C]$ satisfies the first-order condition

$$K |P'_{\text{breach}}(\varepsilon^{**})| = \lambda_\varepsilon(\varepsilon^{**}), \quad (4)$$

i.e. marginal deployment benefit equals marginal robust price. An interior optimum exists iff $K|P'_{\text{breach}}(0)| > \lambda_0$ (the *true* marginal cost at 0^+ , an over-confident-regime quantity, not the margin-density-window bound), is unique, and satisfies $\varepsilon^{**} \leq \varepsilon_{\text{cert}}$. The solver **optimal_radius** brackets and roots the first-order condition (4); Figure 1(c) shows an interior ε^{**} . The reading is an *insurance* one: coverage is ε , the premium is λ_ε , and one buys robustness up to the point where the marginal premium equals the marginal reduction in expected breach loss.

Fitting P_{breach} (concrete example). The breach model is not assumed—it is fit to a few red-team points. Given attack-success rates at several radii (e.g. from AutoAttack), **fit_breach_model** returns a smooth P_{breach} and its derivative, which feed the solver directly:

```
# red-team evaluations: attack success (breach prob) at several radii
eps_pts = [0.0, 0.05, 0.1, 0.2, 0.3, 0.5]
successes = [0.90, 0.74, 0.58, 0.36, 0.21, 0.09]
fit = fit_breach_model(eps_pts, successes, family="logistic") # rmse ~ 0.04
res = optimal_radius(price_fn, fit["Pbreach"], K=2.0, eps_max=eps_cert,
                    Pbreach_prime=fit["Pbreach_prime"])
res.eps_star # -> the budget to train to
```

The fit is robust to the functional form (logistic or exponential); only monotonicity and the slope near 0 affect existence of an interior ε^{**} . Figure 4 runs exactly this on the trained MLP.

When is ε^{} safe to trust? A pre-flight checklist.** The decision calculus is only as good as the price and breach inputs. Before acting on ε^{**} , verify the gates of Table 1; if any fails, the number is not trustworthy and the fault is localized. This consolidates the per-instrument cookbooks into one gate.

gate (must hold)	if it fails
operating point in margin-density window ($ t \lesssim M$; robust acc. bounded from 0, 1)	price is in the tail / over-confident regime; reading uninformative—move the operating point
collapse residual under its regime floor ($\lesssim 5\%$ closed-form, $\lesssim 30\text{--}40\%$ empirical KDE)	meter, k_ε , and operating point are inconsistent—do not price
curvature bracket tight ($\kappa_\infty^\uparrow/\kappa_\infty^\downarrow \approx 1$) at the target ε	first-order proxy unreliable—shrink ε or use finite differences
price CI narrow (95% width $\lesssim 10\%$ of the mean)	undersampled—enlarge the batch (§3)
breach curve monotone decreasing with $K P'_{\text{breach}}(0) > \lambda_0$	no interior optimum (corner); robustness not worth buying, or buy to $\varepsilon_{\text{cert}}$
extrapolation below the measurement wall	re-calibrate the retraining-free proxy before trusting the price schedule

Table 1: Reliability checklist for the decision calculus. All gates green $\Rightarrow \varepsilon^{**}$ is trustworthy; any red localizes the fault. The collapse residual (row 2) carries a regime-dependent threshold (§7).

Cookbook. The two inputs you must supply are $P_{\text{breach}}(\varepsilon)$ and K . Estimate P_{breach} from your deployment threat model—red-team success rates at a few radii, or historical incident frequencies—and fit a smooth decreasing curve; the result is insensitive to its exact shape (only monotonicity and the slope at 0 matter for existence). Set K to the actual cost of a single breach in whatever unit your value function uses (dollars, reputation, downtime). Then read the outcome as a decision: an *interior* ε^{**} is your budget; a *corner* at 0 means robustness is not worth buying against this threat model (the marginal premium exceeds the marginal benefit even at $\varepsilon \rightarrow 0$); a corner at $\varepsilon_{\text{cert}}$ means buy all the way to the certification limit. Use the comparative statics as a planning tool: ε^{**} falls as model complexity (d , or r_{eff}) rises, quantifying the folklore that “a bigger model is economically harder to robustify.” Feed the solver *measured* prices from the meter, not the population floor, so the budget reflects your actual model.

5 Closed-Loop Resource Control (control)

Decision feeds control. Rather than solve (4) offline, the controller drives the training-time budget to ε^{**} online, using only a measured price λ_ε and the deployment marginal benefit $K|P'_{\text{breach}}(\varepsilon)|$:

$$\varepsilon_{t+1} = \text{clip}(\varepsilon_t + \eta F'(\varepsilon_t), 0, \varepsilon_{\text{max}}), \quad F'(\varepsilon) = K|P'_{\text{breach}}(\varepsilon)| - \lambda_\varepsilon. \quad (5)$$

Because the net value is strictly concave in the relevant range, F' is strictly decreasing and the damped fixed-point iteration converges to ε^{**} (a damped-Newton variant is also provided). The controller (**BudgetController**) never retrains at each candidate radius—it consumes measured prices. Figure 2(a) shows convergence to the same ε^{**} from three initializations, matching the offline solver.

Cookbook. Start the gain η small and increase it until convergence is fast but non-oscillatory; if F' is smooth, the damped-Newton mode reaches ε^{**} in a handful of steps. Always clip to $[0, \varepsilon_{\text{cert}}]$ —there is no reason to spend past the certification limit. Drive the loop with *measured* prices from the meter (Instrument 1), and gate on the collapse residual (Instrument 4): if the residual trips, freeze the controller until the regime is re-established, since a mis-measured price will steer the budget wrong. Declare convergence on the first-order residual $|F'(\varepsilon)|$, not on the step size alone. When to prefer the controller over the one-shot solver: use the solver for a static deployment decision, and the controller when the price schedule drifts during training (feature learning reshapes k_ε , hence λ_ε) so the target ε^{**} is itself moving.

6 A Worked Example: From Measurement to Budget

The instruments compose into a short workflow. Suppose a team is deploying a classifier against an ℓ_∞ threat and must decide a training robustness budget. The session proceeds in four moves, each one instrument, and mirrors the validation of §8.

Move 1 — measure and trust. Read the price at the current operating point with one backward pass, $\hat{\lambda}_\varepsilon = \mathbb{E} \|\nabla_x \ell\|_1$, and its bootstrap CI (Instrument 1). Before believing it, run two guards: confirm the

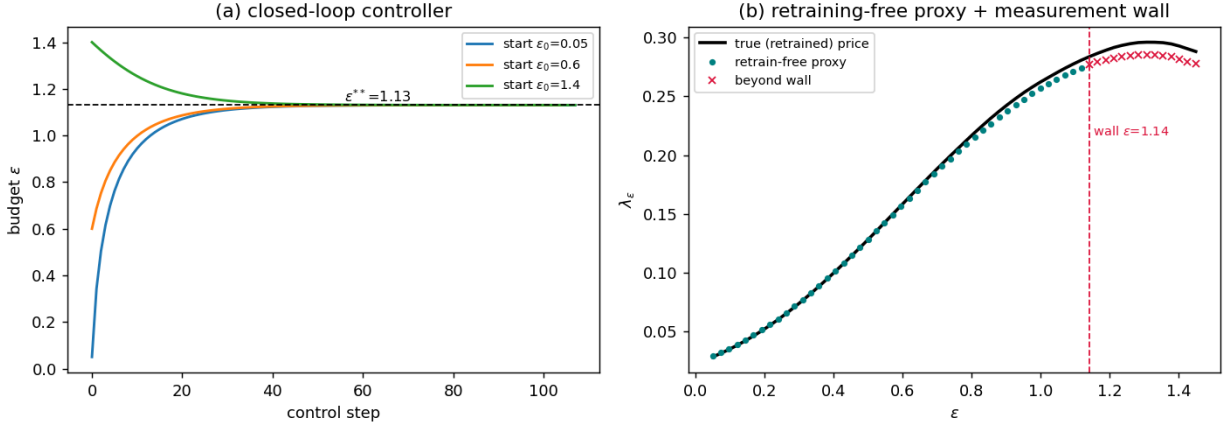


Figure 2: Control and retraining-free measurement (Stage A). (a) The closed-loop controller (5) drives the budget to $\varepsilon^{**} = 1.13$ from three initializations, matching the offline `optimal_radius` solver. (b) The retraining-free proxy, fed a *noisy single-pass* gradient measurement, reproduces the true (retrained) price to $\sim 2\%$ below the measurement wall ($\varepsilon \approx 1.14$) and is correctly flagged (crosses) as it diverges beyond it.

operating point is in the margin-density window (robust accuracy bounded away from 0 and 1), and check the dimensionless-collapse residual (Instrument 4) is small—if it is, the meter, the effective support, and the operating point are mutually consistent and the price is trustworthy. On the synthetic model of §8 the residual sits at $\sim 10^{-6}$; on a real model one would accept, say, $< 5\%$.

Move 2 — locate the regime. Read the effective support k_ε (Instrument 3): its value relative to d tells you which tax you are paying ($\sqrt{d}$, $\sqrt{k}$, or $\sqrt{r_{\text{eff}}}$), and tracking it over training tells you whether feature learning is contracting or delocalizing the gradient. If a finite- ε price is needed rather than the $\varepsilon \rightarrow 0$ proxy, bracket the curvature κ_∞ (Instrument 2); a tight bracket certifies the proxy, a wide one says shrink ε or take a finite difference.

Move 3 — extrapolate cheaply. Rather than retrain a robust model at every candidate radius, calibrate the retraining-free proxy once (Instrument 5) and read the whole λ_ε curve up to the measurement wall. This is what makes the next move affordable: the decision calculus needs the price *schedule*, not a single point.

Move 4 — budget and steer. Supply the deployment breach model P_{breach} and the single-breach loss K , and solve for ε^{**} (§4); if the price schedule drifts during training, hand the same measured prices to the controller (§5) to track the moving optimum online. The output is a defensible number—“train to radius ε^{**} ”—with an audit trail: the price that set it, the regime it was measured in, and the breach model that valued it.

The whole workflow is four cheap readouts and one root-find; none of it requires a robustness sweep, and every step carries a self-check that trips before the answer goes wrong.

Composing with attack suites (a concrete pipeline). `shadowprice` does not attack or benchmark; it consumes the outputs of tools that do. A concrete end-to-end pipeline: (1) run AutoAttack (Croce & Hein, 2020) (or a RobustBench (Croce et al., 2021) evaluation) at a handful of radii to obtain empirical robust accuracies—i.e. the breach curve $\{(\varepsilon_i, \text{success}_i)\}$; (2) fit it with `fit_breach_model` to get a smooth P_{breach} (Move 4); (3) read the marginal price with the meter; (4) solve for ε^{**} and steer with the controller. The attack suite answers “how robust is the model at radius ε ”; `shadowprice` turns that evaluation into “what does robustness cost and how much to buy.” The two are complementary layers of one workflow—evaluation feeds pricing—not competitors; §7 runs exactly this pipeline (with PGD standing in for AutoAttack) on a trained network.

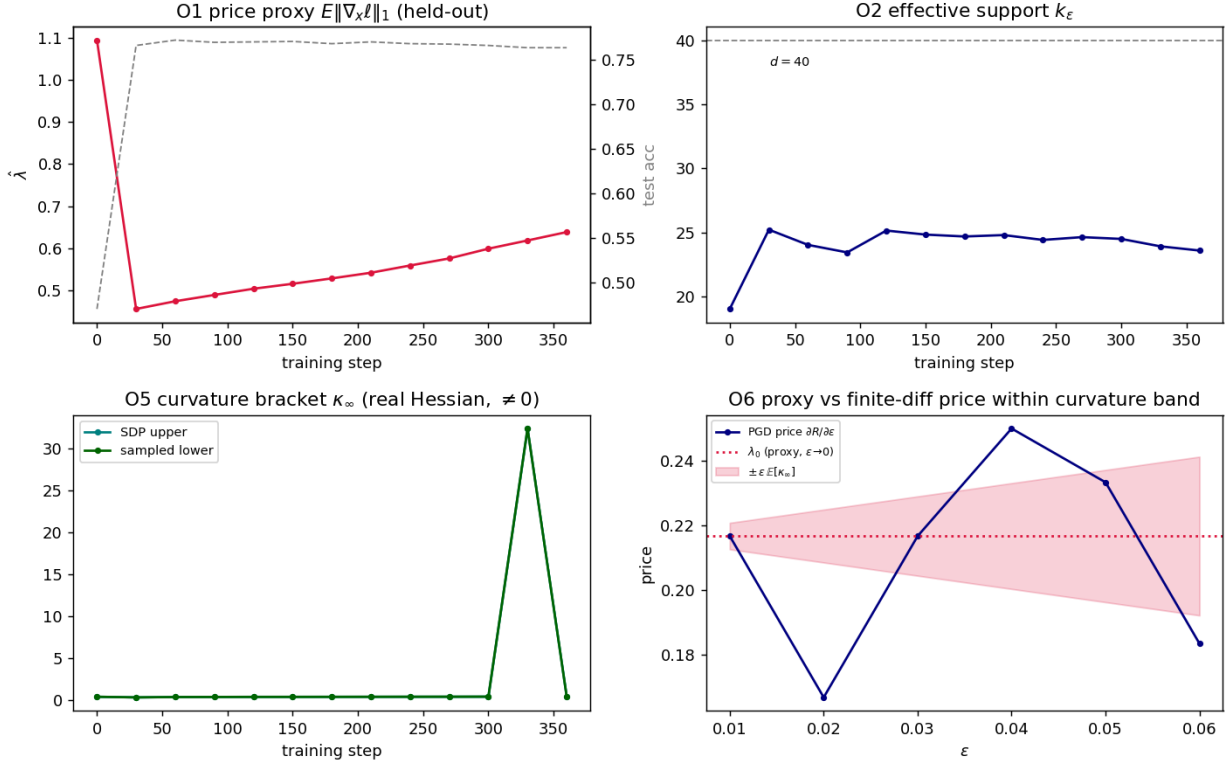


Figure 3: The panel on a trained MLP (held-out measurement). **Top-left:** the price proxy $E\|\nabla_x \ell\|_1$ falls as the model learns (test accuracy rises, dashed). **Top-right:** the effective support k_ϵ evolves in $(1, d)$ with $d = 40$. **Bottom-left:** the curvature bracket κ_∞ from a real (finite-difference) input Hessian—*nonzero*, unlike the linear model (O5). **Bottom-right:** the finite-difference PGD price stays within the $\pm \epsilon E[\kappa_\infty]$ curvature band of the $\epsilon \rightarrow 0$ proxy in the first-order neighborhood (O6), confirming the train-inference bridge; the residual is finite-sample PGD noise.

7 Stage A.5: A Small Trained Network

Between the closed-form checks (Stage A) and deep networks at scale (Stage B) sits an intermediate evidence tier we call *Stage A.5*: a real, trained, nonlinear model small enough to run exactly and instrument fully on a CPU. We run the panel on an *actual trained nonlinear model*: a one-hidden-layer MLP (pure numpy, $d = 40$, 8 hidden units) trained by gradient descent with weight decay on a low-SNR binary task, with all instruments measured on a *held-out* test set so the operating point stays in the margin-density window (test accuracy ≈ 0.76 , not interpolated). Everything uses the model’s true input gradients and a finite-difference input Hessian—no closed forms. This is a small CPU model, not a deep network at scale (see the evidence hierarchy, Table 3); it is the honest next rung above the synthetic validation, and it populates the two observables (O5 curvature, O6 gap) that are trivially zero on the linear model.

The finite-difference PGD price, precisely. On a nonlinear model the “true” price has no closed form, so we define a measurable reference. Let $R_{\text{PGD}}(\epsilon)$ be the 0/1 robust error of the model under a k -step ℓ_∞ PGD attack of radius ϵ (here $k = 15$, step $2.5\epsilon/k$, no restarts, evaluated on the $n = 2000$ held-out points). The *finite-difference PGD price* is the central difference

$$\hat{\lambda}_\epsilon^{\text{PGD}} = \frac{R_{\text{PGD}}(\epsilon + h) - R_{\text{PGD}}(\epsilon - h)}{2h}, \quad h = 0.02.$$

Two caveats make the comparison honest: (a) PGD *under*-attacks, so $R_{\text{PGD}} \leq R$ and $\hat{\lambda}^{\text{PGD}}$ is a lower estimate of the true price (a stronger suite such as AutoAttack tightens it); (b) R_{PGD} is a Monte-Carlo 0/1

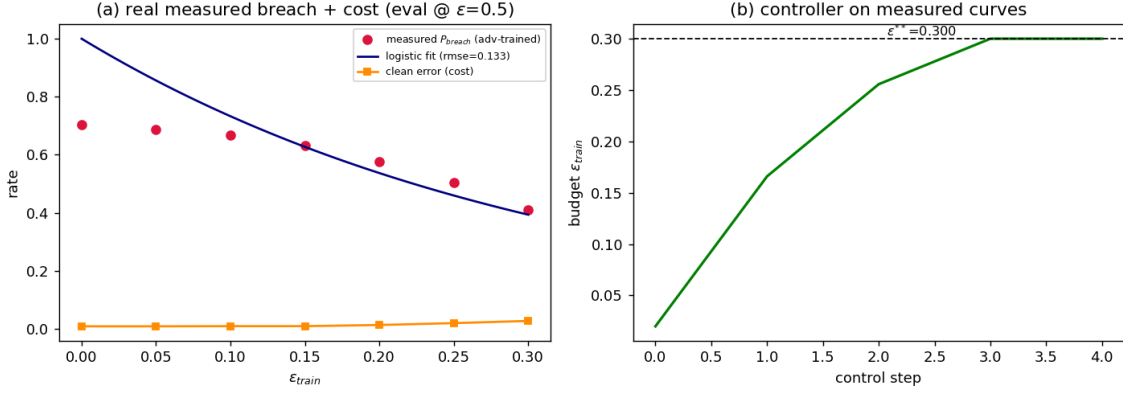


Figure 4: Budgeting on the trained MLP with a *real measured* breach curve. **(a)** On a task with a robustness–accuracy tradeoff (a robust feature plus non-robust ones, Tsipras-style), we adversarially train the MLP at seven radii and *measure* the resulting breach rate $P_{\text{breach}}(\epsilon_{\text{train}})$ at a fixed evaluation attack ($0.70 \rightarrow 0.41$, logistic fit rmse 0.13) and its cost, the clean-error increase ($0.01 \rightarrow 0.03$)—not a synthetic curve. **(b)** Feeding the measured breach and cost curves to the decision calculus, the optimal budget coincides with the measured breach minimum ($\epsilon^{**} \approx 0.30$, a boundary here because robustness is cheap on this task), to which the controller converges. Because ϵ^{**} is at the boundary, the interior noise-robustness numbers (§8) are reported on the Stage-A optimum instead.

average, so the difference carries noise $\propto \sqrt{p(1-p)/n}/h$, which dominates the O6 residual at the smallest radii. The proxy $\mathbb{E}\|\nabla_x \ell\|_1$ is the independent $\epsilon \rightarrow 0$ estimate; O6 checks the two agree within the $\epsilon \mathbb{E}[\kappa_\infty]$ curvature band.

Figure 3 shows the four ready instruments over training. Three findings. **(i)** The price proxy and effective support behave as the theory predicts: as the model fits, the input-gradient price falls and k_ϵ stays a genuine effective dimension in $(1, d)$. **(ii)** The curvature κ_∞ is now nonzero (real Hessian), and the train–inference gap (O6) between the one-pass proxy and the finite-difference PGD price stays within the $\epsilon \mathbb{E}[\kappa_\infty]$ band in the first-order neighborhood—the bridge of §2 holds on a real model, up to finite-sample noise. **(iii)** The empirical margin-density identity (Instrument 4) predicts the measured price with a bootstrap median residual of $\sim 17\%$ (IQR 7–24%, 60 resamples) using only a KDE of the model’s own margins—no Gaussian assumption—demonstrating the collapse check outside closed forms. This is *within* the appropriate threshold: the tight $\approx 5\%$ target applies only where a closed form makes the identity exact (Stage A); the empirical KDE form carries an intrinsically higher floor from finite-sample density estimation and gradient–margin heterogeneity, for which a looser $\approx 30\text{--}40\%$ band is the healthy range (Table 1). The residual here is a passing self-check, not a failure.

Figure 4 runs the full decision pipeline of §6 on the trained model with a *real measured* breach curve. On the clean Gaussian task above, adversarial training cannot trade accuracy for robustness—the Bayes classifier is already robust—so we use a task with a genuine robustness–accuracy tradeoff (a robust feature plus many non-robust ones), adversarially train the MLP at seven radii, and *measure* $P_{\text{breach}}(\epsilon_{\text{train}})$ ($0.70 \rightarrow 0.41$) and its cost, the clean-error increase ($0.01 \rightarrow 0.03$). Feeding these measured curves to the calculus, the optimal budget lands at the measured breach minimum ($\epsilon^{**} \approx 0.30$), reached by the controller. This is the measure→decide→control loop end-to-end on a real (if small) network with no synthetic inputs. The end-to-end trace (real output) is reproduced in Listing 1.

Listing 1: End-to-end trace on the numpy MLP (verbatim output of `demo_end_to_end.py`; no number is fabricated).

```
[model] held-out test accuracy = 0.763 (in margin-density regime)
[measure] price proxy  $\mathbb{E}\|\text{grad}_x \text{loss}\|_1 = 0.639$  (95% CI [0.602,0.674])
[measure] effective support  $k_{\epsilon} = 23.6$  /  $d=40$ 
[measure] curvature bracket  $\kappa_{\text{inf}} = [0.410, 0.410]$  (ratio 1.00; trustworthy)
```

	measurement model (panel)		budget model (breach curve)
architecture	MLP, $d=40$, 1 hidden ($h=8$), ReLU	task	robust + 38 non-robust feats
task	Gaussian mixture, noise $\sigma=8$ (low SNR)	architecture	MLP, $h=16$
train / test	2000 / 2000, held-out measurement	adv. training	PGD 7-step, radii 0–0.3 (7 models)
optimizer	GD, lr 0.15, wd 5×10^{-3} , 360 steps	eval attack	PGD 15-step at $\epsilon=0.5$
seeds	train 1, test 2, model 0	measured	P_{breach} , clean-error cost

Table 2: Stage A.5 reproducibility. Both models are pure numpy; scripts `demo_mlp_training.py` (panel + budget) and `demo_end_to_end.py` (trace) regenerate all numbers deterministically. Hardware: single-core Intel Xeon @ 2.10 GHz, numpy 2.4; wall-clock ≈ 19 s (panel + budget, incl. 7 adversarial trainings) and ≈ 26 s (benchmark, Appendix B).

tier	model	establishes	does <i>not</i> establish
A (done)	linear–Gaussian, closed forms	estimators are correct (13/13 exact checks)	transfer to nonlinear/real models
A.5 (done)	small trained MLP (CPU)	instruments run & cohere on a real model; O5/O6 populate	deep-network-scale behavior
B (specified)	deep nets + AutoAttack-/RobustBench	(to be tested) real-scale accuracy & the falsification checks	— (not run here)

Table 3: Evidence hierarchy. This paper delivers Stages A and A.5 and specifies Stage B (§8, protocol below) with a runnable template (`torch_resnet_pricing.py`).

```
[decide] measured breach P_breach(eps_train) = 0.70 -> 0.41 (7 adv-trained models); fit rmse 0.133
[decide] optimal robustness budget eps** = 0.300
[control] closed-loop controller -> 0.300
```

8 Validation

We separate three tiers of evidence and are explicit about what each does and does not establish (Table 3). **Stage A** is closed-form validation: on the linear–Gaussian model every instrument is checked against an exact formula—this establishes the estimators are *correct*, not that they transfer. **Stage A.5** (§7) is a small trained nonlinear model: real gradients, a real Hessian, an in-regime operating point—this establishes the instruments *run and cohere on a real model*, at CPU scale, but not at deep-network scale. **Stage B** is deep networks at scale under a strong attack suite: specified with falsification conditions and a runnable template, but *not executed here*.

Stage A (synthetic, completed). On the linear–Gaussian model where all quantities have closed forms, every instrument matches its Paper A prediction. The finite-difference meter matches (2) to $\sim 10^{-13}$ (Figure 1a); the curvature bracket is validated against a brute-force Boolean maximum for small d , with the SDP bound tighter than the trivial one; the controller reaches the solver’s ϵ^{**} from both sides (Figure 2a) and stays within a tight tolerance of the interior optimum $\epsilon^{**}=1.13$ under injected price noise: the tail-averaged $|\epsilon - \epsilon^{**}|$ is 0.0007, 0.0023, and 0.0051 at 5%, 15%, and 30% multiplicative noise respectively (*i.e.* $< 0.5\%$ relative even at 30% noise); the retraining-free proxy reproduces the retrained price below the wall (Figure 2b); and the six Stage-A observables—price, k_ϵ , collapse residual (3), ϵ_{cert} , curvature ratio, and the train–inference gap—behave as predicted, with the dimensionless-collapse residual at $\sim 10^{-6}$ across the sweep (Figure 5). All checks ship as an automated suite (13/13 passing), enumerated in Table 7.

Estimator cost and guarantees. Table 4 summarizes what each instrument costs and what kind of guarantee it carries—a practitioner needs to know both. Table 8 reports measured wall-clock times (Figure 6): the gradient-based instruments and the effective support are essentially free (< 1 ms even at $d=1000$), the

instrument	cost (beyond gradients)	d -scaling	guarantee
price proxy $\mathbb{E}\ \nabla_x \ell\ _1$	1 backward pass/batch	$O(bd)$	$\varepsilon \rightarrow 0$ exact; $O(\varepsilon)$ error at finite ε
finite-diff price	$2\times$ risk eval	$O(1)$	unbiased + $O(h^2)$ bias, δ^{-3} budget
k_ε (participation ratio)	elementwise	$O(bd)$	exact
κ_∞ (abs bound)	$\sum H_{ij} $	$O(d^2)$	valid <i>upper</i> bound
κ_∞ (SDP)	convex SDP	$O(d^3)$	tighter upper bound (audit)
κ_∞ (sampling)	random signs + 1-opt	$O(d^2 n_s)$	valid <i>lower</i> bound
$\varepsilon_{\text{cert}}, \varepsilon^{**}$	closed form / 1-D root	$O(1)$	exact / FOC-exact
retraining-free proxy	1 calibration batch	$O(bd)$	exact (linear) / approx. to wall

Table 4: Estimator costs and guarantees. “Exact” means it recovers the Paper A quantity; “upper/lower” means a certified one-sided bound; “approx.” means validated empirically, not guaranteed.

	observable	cost	Stage-A result / Stage-B check
O1	price λ_ε	1 backward pass	matches (2) to $\sim 10^{-13}$ / within $\varepsilon \mathbb{E}[\kappa_\infty]$ band
O2	effective support k_ε	1 backward pass	$\in (1, d)$, tracks soft-threshold / scales as $\sqrt{k_\varepsilon}$ not $\sqrt{d}$
O3	collapse residual (3)	free (self-check)	$\sim 10^{-6}$ / stays small in-regime
O4	cert radius $\varepsilon_{\text{cert}}$	closed form	exact / matches empirical chance-crossing
O5	curvature ratio	SDP audit	bracket valid (brute-force) / widens where proxy fails
O6	train–inference gap	1 pass + 1 probe	$\leq \varepsilon \mathbb{E}[\kappa_\infty]$ / bounded by curvature

Table 5: The six Stage-A observables: each is cheap, tied to a Paper A closed form, and carries a Stage-B falsification check. O5/O6 are trivial on the linear model ($\nabla^2 f \equiv 0$) and populate for curved models.

certification/budget solves are $O(1)$, the cheap curvature bound is $O(d^2)$, and only the SDP curvature bound is expensive ($O(d^3)$, seconds at $d=200$)—confirming it is an audit tool, not a per-step monitor.

Stage B (real networks, protocol). Stage A validates the *math*; Stage B validates the *measurement* on real models and is deliberately specified but not run here (it requires GPU training). The protocol, and its falsification conditions, are:

1. *Proxy accuracy.* On a moderate-scale classifier, the one-pass proxy $\mathbb{E}\|\nabla_x \ell\|_1$ should match the finite-difference PGD price at small ε within the $\varepsilon \mathbb{E}[\kappa_\infty]$ band. **Falsified if** the gap exceeds the curvature bound outside the margin-density window.
2. *Effective-support scaling.* Across dense/sparse/low-rank feature regimes, the measured price factor should scale as $\sqrt{k_\varepsilon}$, with k_ε the gradient participation ratio. **Falsified if** the price scales as $\sqrt{d}$ when $k_\varepsilon \ll d$.
3. *Curvature wall.* The proxy’s error should grow where the κ_∞ bracket widens. **Falsified if** the proxy stays accurate where curvature is large, or fails where it is small.

These three are the *core* Stage-B validation. A fourth, *optional and exploratory*, check tests the effective-rank hypothesis and is not required for the core measurement claims:

4. *(Optional) Effective-rank lever.* A LoRA-adapted model with small r_{eff} should show $k_\varepsilon \approx \Theta(r_{\text{eff}})$ under the occupancy conditions. **Falsified if** k_ε tracks d regardless of r_{eff} .

The library’s PyTorch adapter (`adapters.torch`) supplies the `grad_fn/risk_fn` (with a minimal PGD loop) needed to execute this protocol, and the reference script `torch_resnet_pricing.py` implements the full real-model workflow end-to-end: train a network, obtain the breach curve from an attack suite (AutoAttack/RobustBench, with PGD as a drop-in fallback), fit P_{breach} , price with the meter and an HVP-based curvature probe, and solve for ε^{**} . It is a complete, runnable template—not executed in this paper (it needs a GPU/torch), so we report no numbers from it, keeping the evidence hierarchy of Table 3 clean.

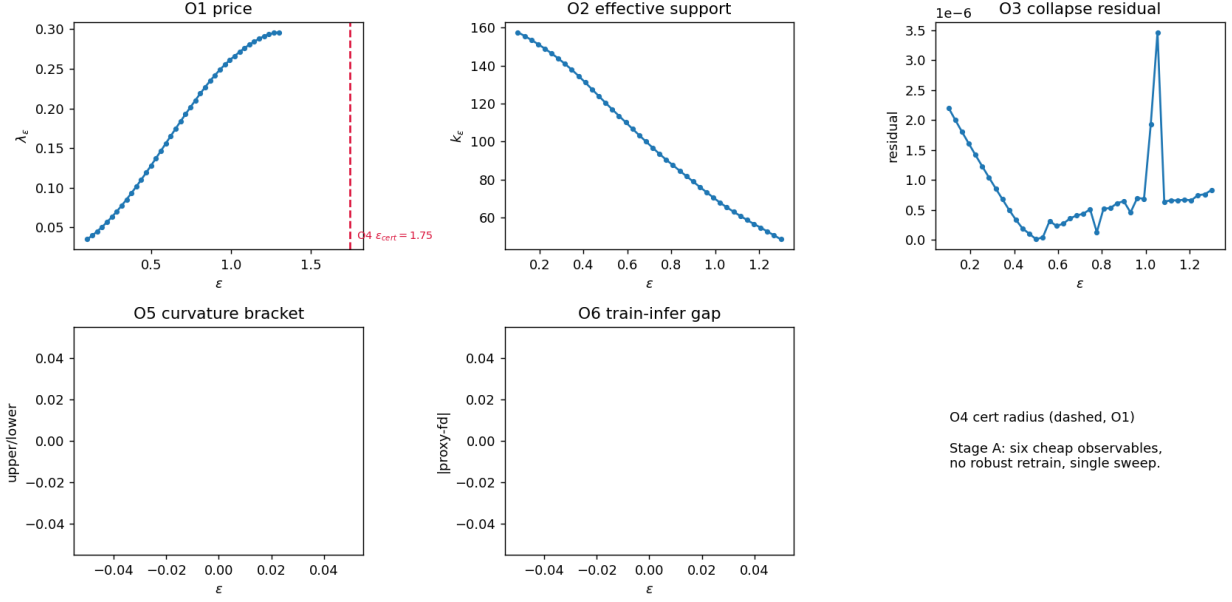


Figure 5: The six low-cost Stage-A observables over an ε -sweep (dashboard). O1 price, O2 effective support k_ε , O3 dimensionless-collapse residual ($\sim 10^{-6}$, confirming the scale-invariance prediction $\lambda_\varepsilon \sigma / \phi(t) = \sqrt{k_\varepsilon}$), and O4 the certification radius (dashed). O5 (curvature bracket) and O6 (train-inference gap) are empty here because the linear model has $\nabla^2 f \equiv 0$; they populate on the trained MLP of §7.

9 The Library

`shadowprice` is organized as the three layers above plus a reporting layer:

layer	module	contents
measure	<code>price_meter</code>	$\mathbb{E} \ \nabla_x \ell\ _1$ proxy, finite-difference price, δ^{-3} budget
	<code>kappa_infty</code>	SDP + dependency-free upper bounds, random-sign lower bound
	<code>effective_support</code>	k_ε diagnostic, training-time tracker
decide	<code>proxy</code>	retraining-free extrapolation + measurement wall
	<code>radius</code>	$\varepsilon_{\text{cert}}$, ε^{**} solver, comparative statics
control	<code>controller</code>	closed-loop budget controller (proportional / damped-Newton)
report	<code>diagnostics</code>	the six Stage-A observables + dashboard
adapt	<code>adapters.torch</code>	PyTorch <code>grad_fn/risk_fn</code> (optional, lazy import)

The core has no heavy dependencies: `cvxpy` is optional (only the tightest curvature bound needs it) and `torch` is optional (only the adapter). A minimal session:

```
import shadowprice as sp
k = sp.participation_ratio(grad_x) # effective dimension in [1, d]
out = sp.price_gradnorm(grad_fn, X) # one-pass price + bootstrap CI
br = sp.kappa_infty(H) # curvature bracket {lower, upper}
ec = sp.cert_radius(w, mu) # certification radius
res = sp.optimal_radius(price_fn, Pbreach, K, eps_max=ec) # res.eps_star
ctl = sp.BudgetController(price_fn, marginal_benefit, eps_max=ec)
eps_star = ctl.run(eps0=0.1).eps_star # closed-loop to eps**
```

Design principles. Three choices keep the toolkit usable and trustworthy. *Framework-agnostic core:* every estimator consumes plain callables, so the same code prices a numpy toy, a PyTorch classifier, or a JAX model; the framework enters only through a thin adapter. *Optional heavy dependencies:* nothing in the import path requires `cvxpy` or `torch`—the curvature bound degrades gracefully to a dependency-free formula, and the adapter is imported lazily—so the library installs and runs in a minimal environment, which matters for a diagnostic that should be easy to drop into an existing stack. *Validated against closed forms, and self-checking at runtime:* each instrument is unit-tested against a Paper A closed form (the 13/13 suite), and the dimensionless-collapse residual gives the *user* a runtime self-check, so a mis-measurement trips a visible switch rather than silently corrupting a budget. The library is small on purpose: it is a measurement instrument, not a training framework.

Artifact and reproducibility. The toolkit is released as open source under the **Apache-2.0** license at <https://github.com/<anon>/shadowprice> (anonymized for review). Installation is `pip install shadowprice` (core: `numpy`, `scipy`), with optional extras `[sdp]` (`cvxpy`, for the tightest curvature bound), `[torch]` (the PyTorch adapter), and `[plots]`. The test suite (`pytest`) runs the 13 checks of Table 7 against Paper A closed forms; the scripts `demo_instrument_panel.py`, `demo_control_diagnostics.py`, `demo_mlp_training.py`, and `benchmark.py` regenerate Figures 1–6 deterministically (all seeded, CPU, versions pinned in the README). The Stage-B reference `torch_resnet_pricing.py` is a complete, runnable template that requires PyTorch and is *not* executed in the paper build; no reported number depends on it.

10 Related Work and Limitations

Robustness tooling. Existing open-source toolkits—attack libraries and benchmarks such as Foolbox (Rauber et al., 2017), AutoAttack (Croce & Hein, 2020), and RobustBench (Croce et al., 2021), building on FGSM/PGD (Goodfellow et al., 2015; Madry et al., 2018)—answer “how robust is this model?” by *attacking* and *evaluating*. They do not *price* robustness: none reads the marginal cost λ_ϵ , its pricing unit k_ϵ , or the optimal budget ϵ^{**} . `shadowprice` is complementary—a *measurement and decision* layer, not an attack or benchmark.

Certified robustness. Certification methods such as randomized smoothing (Cohen et al., 2019) produce a per-input certified radius; our ϵ_{cert} is a coarse population analogue under the binary-channel reduction, and ϵ^{**} adds the economic layer (how much certified radius is worth buying) that certification alone does not address.

Limitations. (i) All validation here is Stage A—synthetic and a single small trained model—not a deep network at scale; Stage B is specified but not executed. (ii) The deep-network price is a *local* first-order proxy at a working point (Paper A), valid within the κ_∞ -bounded neighborhood, not a global guarantee. (iii) The optimal-radius result is a reduced-form economic model: P_{breach} is supplied by the deployment threat model, not derived.

Future work: the effective-rank lever. Paper A’s conditional $k_\epsilon = \Theta(r_{\text{eff}})$ suggests low-rank adaptation (LoRA/PEFT, Hu et al., 2022) could lower the tax from $\sqrt{d}$ toward $\sqrt{r_{\text{eff}}}$; we do not test this. It is falsifiable with the effective-support diagnostic—check whether k_ϵ of a LoRA-adapted model tracks r_{eff} rather than d —and we list it as a Stage-B hypothesis, not a result.

11 Conclusion

Paper A proved the marginal price of robustness has a floor set by the effective support $\sqrt{k_\epsilon}$. This paper makes that price *measurable, decidable, and controllable*: a one-backward-pass-per-batch meter, a curvature early-warning, an effective-dimension diagnostic, a retraining-free extrapolator with a measurement wall, a decision calculus for ϵ^{**} , and a closed-loop controller—each tied to a closed form and validated (13/13) on a synthetic model, packaged as an open-source, framework-agnostic library. The immediate next step is Stage B: executing the falsification protocol of §8 on real networks, for which the library already provides

	attack/eval	cert. radius	price λ_ε	budget ε^{**}
Foolbox / AutoAttack	✓			
RobustBench	✓			
Randomized smoothing		✓		
shadowprice (this work)		(coarse)	✓	✓

Table 6: Positioning (Rauber et al., 2017; Croce & Hein, 2020; Croce et al., 2021; Cohen et al., 2019). Existing tooling attacks, evaluates, or certifies; none reads the marginal *price* of robustness, its pricing unit k_ε , or the economically-optimal budget ε^{**} . The layers are complementary: an attack library answers “how robust is this model,” **shadowprice** answers “what does more robustness cost and how much should I buy.”

the hooks. If the instruments survive that test, robustness budgeting becomes an engineering measurement rather than a guess.

A The δ^{-3} finite-difference budget

The finite-difference price meter estimates $\lambda_\varepsilon = R'(\varepsilon)$ by the central difference $D_h = (\hat{R}(\varepsilon + h) - \hat{R}(\varepsilon - h))/(2h)$, where $\hat{R}$ is a Monte-Carlo risk estimate with per-evaluation variance σ_R^2/n (n samples). Two error sources compete. **Bias:** by Taylor expansion, $\mathbb{E}D_h = R'(\varepsilon) + \frac{h^2}{6}R'''(\varepsilon) + O(h^4)$, so the bias is $\frac{h^2}{6}R'''$.

Variance: the two independent evaluations give $\text{Var}(D_h) = \frac{2\sigma_R^2/n}{(2h)^2} = \frac{\sigma_R^2}{2nh^2}$. Hence

$$\text{MSE}(h) = \frac{(R''')^2}{36} h^4 + \frac{\sigma_R^2}{2n h^2}.$$

Minimizing over h , $\frac{d}{dh}\text{MSE} = \frac{(R''')^2}{9}h^3 - \frac{\sigma_R^2}{nh^3} = 0$, gives the variance-optimal step

$$h_\star = \left(\frac{9\sigma_R^2}{n(R''')^2} \right)^{1/6} \propto \left(\frac{\sigma_R^2}{n} \right)^{1/6},$$

and substituting back, $\text{RMSE}(h_\star) = \Theta((R''')^{1/3}(\sigma_R^2/n)^{1/3}) \propto n^{-1/3}$. To reach a target accuracy δ one therefore needs

$$n \sim \frac{\sigma_R^2(R''')}{\delta^3} = \Theta(\delta^{-3}),$$

the characteristic δ^{-3} *budget*: halving the target error costs $8\times$ the samples. This is what **fd_budget** returns, and check 9 of Table 7 verifies the $n \propto \delta^{-3}$ scaling numerically. The practical lesson (Instrument 1’s cookbook) is to pick the loosest δ the decision tolerates rather than chase precision.

B Validation details and benchmarks

Table 7 enumerates the 13 automated checks; Table 8 and Figure 6 report instrument runtimes.

#	check	validates
1	effective-support extremes	$k_\varepsilon = 1$ (localized), $k_\varepsilon = d$ (delocalized)
2	soft-threshold participation	k_ε matches the closed-form active-set ratio
3	certification radius	$\varepsilon_{\text{cert}} = \langle w, \mu \rangle / \ w\ _1$
4	ε^{**} first-order condition	$K P'_{\text{breach}}(\varepsilon^{**}) = \lambda(\varepsilon^{**})$
5	ε^{**} corner case	no interior optimum when robustness too costly at 0
6	κ_∞ bracket (brute force)	lower $\leq$ true $\leq$ upper for small d ; SDP tighter than $\sum H_{ij} $
7	price gradient-norm proxy	$\mathbb{E} \ \nabla_x \ell\ _1$ on a known batch
8	finite-diff price vs closed form	matches $(\sqrt{k_\varepsilon}/\sigma)\phi(t)$ to $< 10^{-4}$
9	δ^{-3} budget scaling	$n_{\text{req}} \propto \delta^{-3}$ (App. A)
10	controller convergence	reaches solver ε^{**} from both sides
11	retraining-free proxy	reproduces retrained price below the wall ($< 10^{-6}$)
12	collapse residual (scale invariance)	$\lambda_\varepsilon \sigma / p_S(t) = \sqrt{k_\varepsilon}$ to $\sim 10^{-6}$
13	controller noise-robustness	converges near ε^{**} under up to 30% injected price noise

Table 7: The 13 automated Stage-A checks, each against a Paper A closed form.

d	partic. ratio	price (boot)	κ_∞ abs	κ_∞ lower	κ_∞ SDP
10	0.03 ms	2.9 ms	0.005 ms	0.6 ms	35 ms
100	0.06 ms	2.8 ms	0.02 ms	12 ms	1.1 s
200	0.09 ms	2.9 ms	0.09 ms	43 ms	5.6 s
1000	0.61 ms	3.1 ms	12 ms	1.2 s	—

Table 8: Measured wall-clock time per call (batch 256, median of 5). The SDP curvature bound dominates and is intended for periodic audits; all other instruments are millisecond-scale. Full curves in Figure 6.

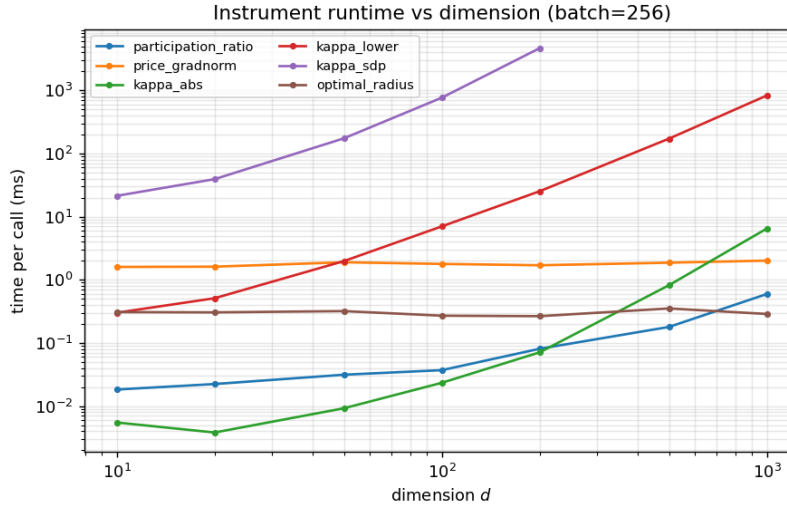


Figure 6: Instrument runtime vs dimension d (log-log, batch 256). The gradient-based instruments, effective support, and the ε^{**} solve are millisecond-scale across the range; the cheap curvature bound is $O(d^2)$; only the SDP bound is expensive ($O(d^3)$), consistent with its role as an audit rather than a continuous monitor.

References

- Jeremy Cohen, Elan Rosenfeld, and J. Zico Kolter. Certified adversarial robustness via randomized smoothing. In *International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pp. 1310–1320, 2019.
- Companion paper (Paper A). The effective-support floor of the adversarial shadow price: An $\omega(\sqrt{k_\varepsilon})$ lower bound. *Companion manuscript*, 2025.
- Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pp. 2206–2216, 2020.
- Francesco Croce, Maksym Andriushchenko, Vikash Sehwal, Edoardo Debenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. RobustBench: a standardized adversarial robustness benchmark. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Jonas Rauber, Wieland Brendel, and Matthias Bethge. Foolbox: A python toolbox to benchmark the robustness of machine learning models. *arXiv preprint arXiv:1707.04131*, 2017.